

PrediSched benchmark s21-autoscale

Generated 2026-09-27 15:55 by scripts/make-report.py from results/benchmark/s21-autoscale/. 9 runs: 1 scenarios × 3 strategies × 3 repetitions.

Headlines: predictive vs the best baseline

Per scenario, the best baseline is the reactive strategy with the best mean for that metric. A difference is only called one at $p < 0.05$ (two-sided Welch t-test over the repetitions; d is Cohen's d). ▲ predictive better, ▼ worse, ● no significant difference.

bursty-autoscale

- Mean latency: no significant difference vs Reactive scaling (-21.4 %, $p = 0.061$)
- p95 latency: no significant difference vs Reactive scaling (-18.4 %, $p = 0.069$)
- Throughput: no significant difference vs Reactive scaling (+2.8 %, $p = 0.265$)
- SLA violations: no significant difference vs No scaling (-42.4 %, $p = 0.167$)

Setup and controls

- Every run starts a fresh cluster of real processes on one laptop (Intel i5-8350U, 4 cores / 8 threads, 15 W): one scheduler (`configs/benchmark-node.yaml`, PostgreSQL history on, result cache off, speculation off) and three workers. The prediction server and the mock HTTP target run for the whole suite.
- Each scenario replays one saved trace from `workloads/benchmark`, identical for every strategy and repetition (SHA-256 checked before every run). Trace seeds were used neither for the training dataset nor for λ tuning.
- Repetition-major order with the strategy order rotated each repetition, so machine drift spreads over strategies.
- The first 10 % of each trace's tasks are excluded from the latency percentiles (warm-up). Every task carries a 2000 ms deadline; SLA violation = failed or later than that.
- Latency is submit → completion as seen by the client (status polled every 50 ms).

Scenario	Profile	Pattern	Rate (/s)	Tasks	Seed	Workers	Fault
steady	mixed	steady	20	240	2001	homogeneous	
bursty	bursty	bursty	12	240	2002	homogeneous	
heterogeneous	mixed	steady	15	240	2003	heterogeneous	
mixed	long_tail	periodic	15	240	2004	heterogeneous	
failure	mixed	steady	15	240	2005	heterogeneous	kill worker-2 at 6 s
bursty-autoscale	bursty	bursty	20	400	2006	single	

Results per scenario

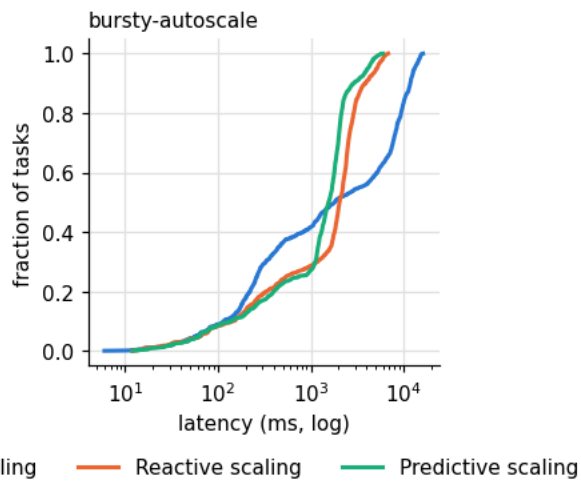
Mean ± standard deviation over the repetitions. The last line gives predictive vs the best baseline for each column.

bursty-autoscale

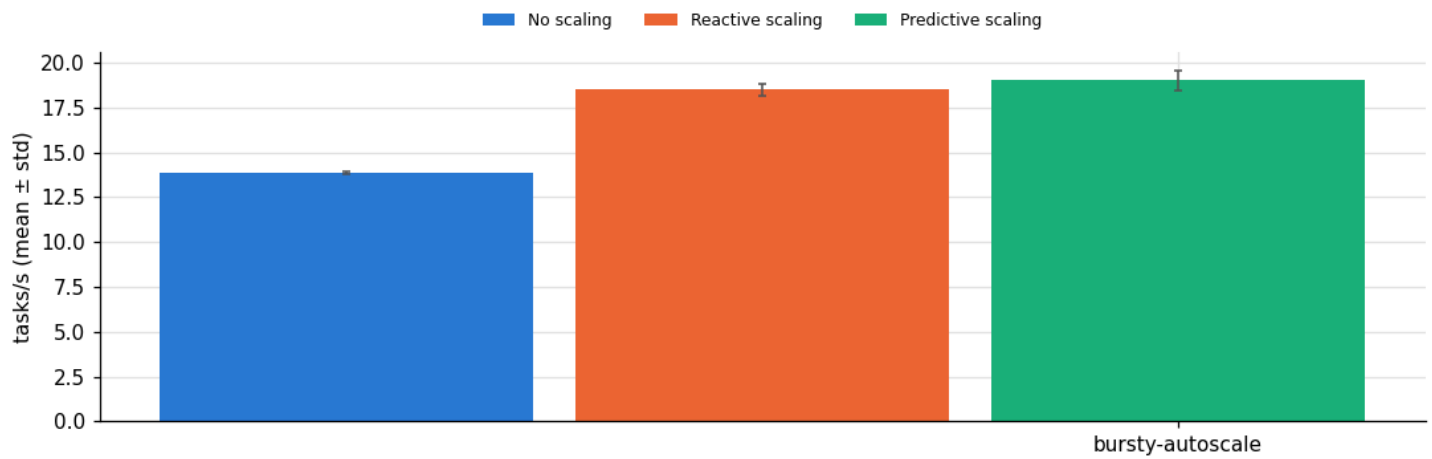
Strategy	Mean latency	p95 latency	p99 latency	Throughput	Makespan	Imbalance (std)
No scaling	4,940 ± 182	13,323 ± 193	15,201 ± 176	13.84 ± 0.09	28,905 ± 186	
Reactive scaling	2,227 ± 211	5,159 ± 518	6,006 ± 481	18.49 ± 0.33	21,643 ± 388	
Predictive scaling	1,750 ± 238	4,209 ± 393	4,991 ± 508	19.00 ± 0.58	21,062 ± 649	
predictive vs best -21.4 % vs Reactive scaling, $p = 0.061$ -18.4 % vs Reactive scaling, $p = 0.069$ -16.9 % vs Reactive scaling, $p = 0.066$ +2.8 % vs Reactive scaling, $p = 0.265$ -2.7 % vs Reactive scaling, $p = 0.269$ +nan % vs						

Latency distribution

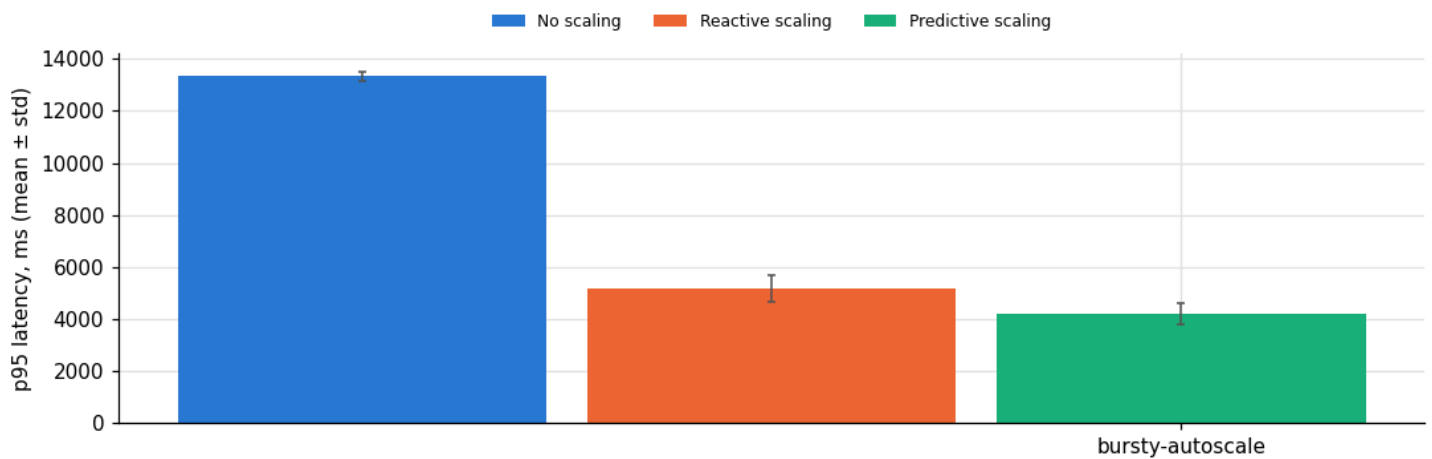
Every completed task of every repetition, per strategy (CDF, log scale).



Throughput

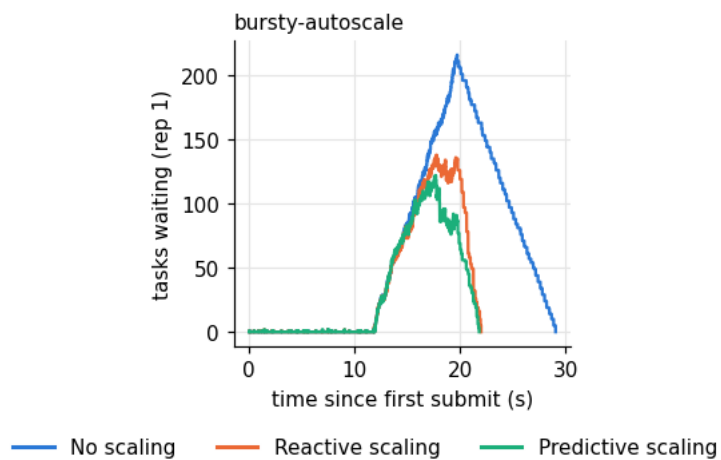


p95 latency

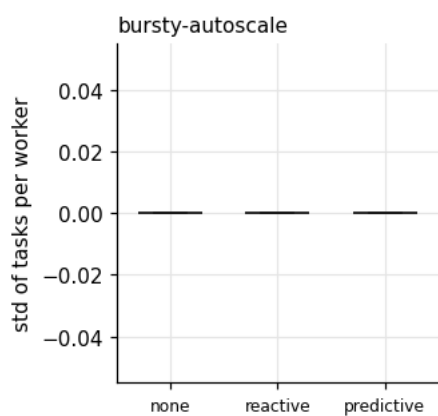


Queue over time

Tasks submitted but not yet started, repetition 1 of each strategy.

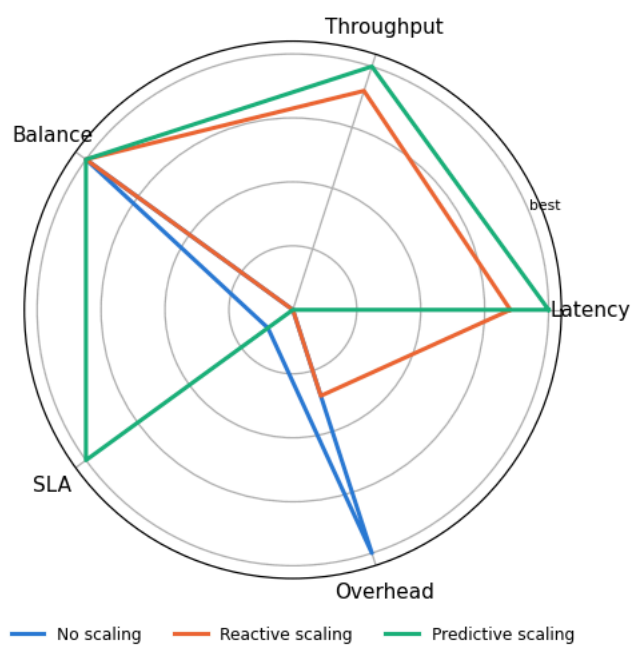


Worker imbalance



Overall profile

Each axis scales the strategies' means to [0, 1] within a scenario (1 = best), averaged over scenarios. A summary, not a test: read the tables for the numbers.



Scheduling overhead and prediction error

Strategy	Decision time mean (µs)	Decision time p95 (µs)
No scaling	40	87
Reactive scaling	58	101
Predictive scaling	67	105

Scenario	Live exec-time MAE of predictive (ms)
bursty-autoscale	n/a

Limitations

- Everything runs on one 4-core laptop: workers share the CPU, so "heterogeneous" hardware is simulated with pool sizes and a slowdown factor, and the prediction server competes with the workers it predicts.
- Five repetitions per cell: small differences can go undetected, and one test per metric and scenario is reported without a multiple-comparison correction.
- The models were trained on one campaign (prompt 15) on this same machine; the queue-forecast model never beat its baseline, and the overload model's threshold did not transfer to the held-out pattern (docs/components/ml-models.md).
- ML predictions can be wrong, and a poor model can make scheduling worse than a simple baseline; prediction calls add overhead to every decision (see above).
- Historical data may not capture sudden, unseen spikes; workload patterns drift, so models need retraining (prompt 21).
- New task types face a cold start (a type-average fallback); monitoring and heartbeats add their own overhead.
- A single active scheduler is a throughput ceiling; these results say nothing about larger clusters.